07. control flow

```
Data Sciences Institute
Python
```

Learning objectives

- Be able to use Boolean values and logical operators to build conditions
- Write conditional statements
- Work with collections of values and apply specific operations
- Use iterations to repeat actions

What is a Boolean?

- A **Boolean** (bool) is a data type with only two possible values:
  - `True`
  - `False`
- Booleans are often used to:
  - Make decisions
  - Compare values → Logic operators are often used with comparisons
    - > greater than
    - < less than
    - == equal to
  - Control program flow (e.g., if statements)

Logical (or Boolean) operators

- Python has 3 main logical operators
  1. `not` → **reverses (or flips)** a Boolean value

x	not x
`True`	`False`
`False`	`True`

💡 Think of *not* as saying **"the opposite of"**

```
# Example evaluates as False
not (3 == 3.0)
```

2. `and` → checks if **both conditions are true**
3. `or` → checks if **at least one condition is true**

Logical (or Boolean) operators

- Python has 3 main logical operators
  1. `not` → **reverses (or flips)** a Boolean value
  2. `and` → checks if **both conditions are true**

x	y	x and y
True	True	True
True	False	False
False	True	False
False	False	False

💡 Think both conditions must be met

```
# Example evaluates as True
7 == 7.0 and 32 > 9
```

3. `or` → checks if **at least one condition is true**

Logical (or Boolean) operators

- Python has 3 main logical operators
  1. `not` → **reverses (or flips)** a Boolean value
  2. `and` → checks if **both conditions are true**
  3. `or` → checks if **at least one condition is true**

x	y	x or y
True	True	True
True	False	True
False	True	True
False	False	False

💡 Think one *or* the other (or both!)

```
# Example evaluates as True
'python' == 'Python' or 32 > 9
```

Best practice: Use Boolean values explicitly

- To avoid confusion and ensure **consistent behaviour**, it is recommended to use **Boolean values** (True or False) explicitly when working with logical operations.
- This helps ensure outcomes are **predictable** and based on clear Boolean logic.
- Python allows non-Boolean values (like numbers or strings) in logical expressions

```
# Example evaluates as 3
7.0 and 3
```

0 is treated as `False`
Any non-zero number is treated as `True`
and returns the **last evaluated value**, not True or False

- This can sometimes lead to **unexpected results** !

Order of precedence

- Boolean operators are evaluated **after** arithmetic and comparison operators!

Order	Operator	Description
1	**	Exponentiation
2	-	Negation
3	*, /, //, %	Multiplication, division, integer division, and modulo
4	+, -	Addition and subtraction
5	<, <=, >, >=, ==, !=	Less than, less than or equal to, greater than, greater than or equal to, equal, not equal
6	not	Not
7	and	And
8	or	Or

Quiz: Logical operators

1. **What is the output of the following expression?**

```
True and False
```

A. True
B. False
C. None
D. Error

Quiz: Logical operators

2. What is the output of the following expression?

```
True or False
```

A. True
B. False
C. None
D. Error

Quiz: Logical operators

3. What does the not operator do?

A. Combines two conditions
B. Checks if two values are equal
C. Reverses a Boolean value
D. Converts a value to a string

Quiz: Logical operators

4. What is the output of the following code?

```
not True
```

A. True
B. False
C. None
D. Error

Quiz: Logical operators

5. What will be printed?

```
age = 20
print(age >= 18 and age <= 25)
```

```
x = 10
print(x < 5 or x == 10)
```

What is control flow?

- **Control flow** refers to the order in which a program's statements are executed
- It determines:
  - Which lines of code run
  - When they run
  - Whether some lines are skipped

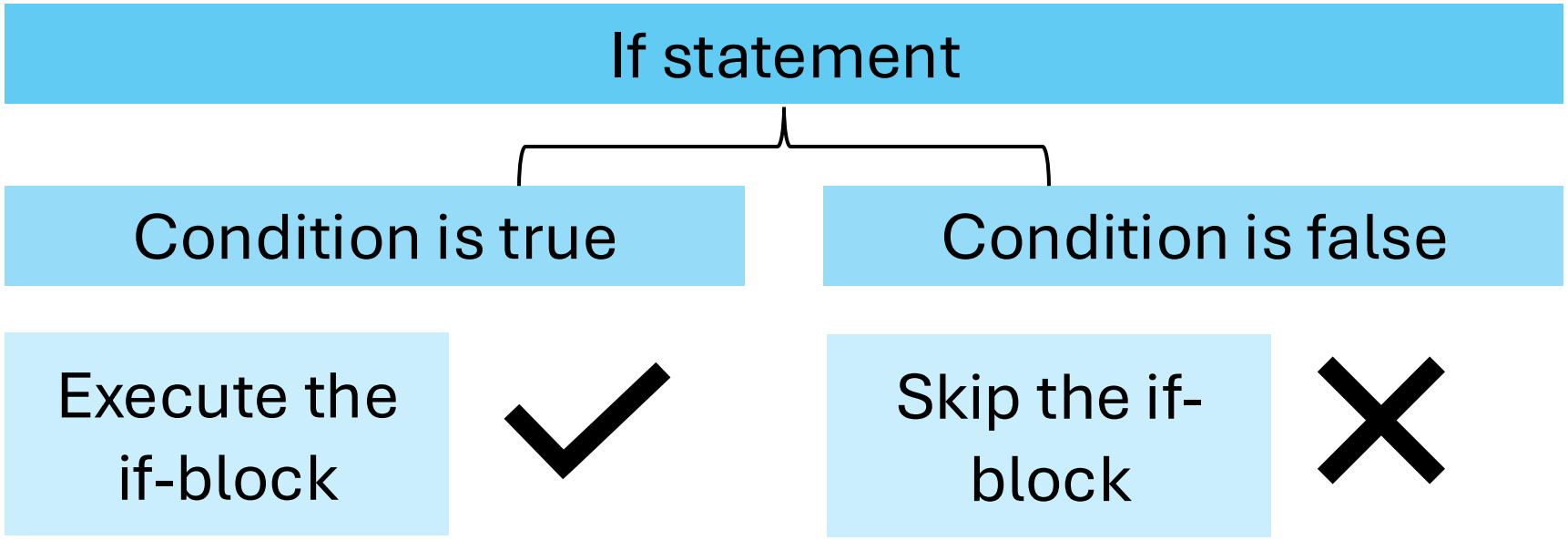
- Control flow allows programs to:
  - Make decisions
  - React to different inputs
  - Run different code in different situations
- Without control flow, programs would always do the exact same thing every time

Conditions

- **A conditional statement** lets Python choose what to do next
- Code runs ONLY if a condition is met
- Condition must evaluate to `True` or `False`

The **if** statement

- Conditional statements start with if
- The if keyword is followed by a condition
- If the condition is `True`, the indented block of code runs
- If the condition is `False`, the block is skipped



The **if** statement

- Conditional statements start with if
- The if keyword is followed by a condition
- If the condition is `True`, the indented block of code runs
- If the condition is `False`, the block is skipped

```
if condition:
    # code that runs if condition is true
```

Indentation matters in Python — it defines the code block

The **if** statement

- Conditional statements start with if
- The if keyword is followed by a condition
- If the condition is `True`, the indented block of code runs
- If the condition is `False`, the block is skipped

```
# Example of basic if statement

year = 2022
if year >= 2000:
    print('We are in the 21st century')
```

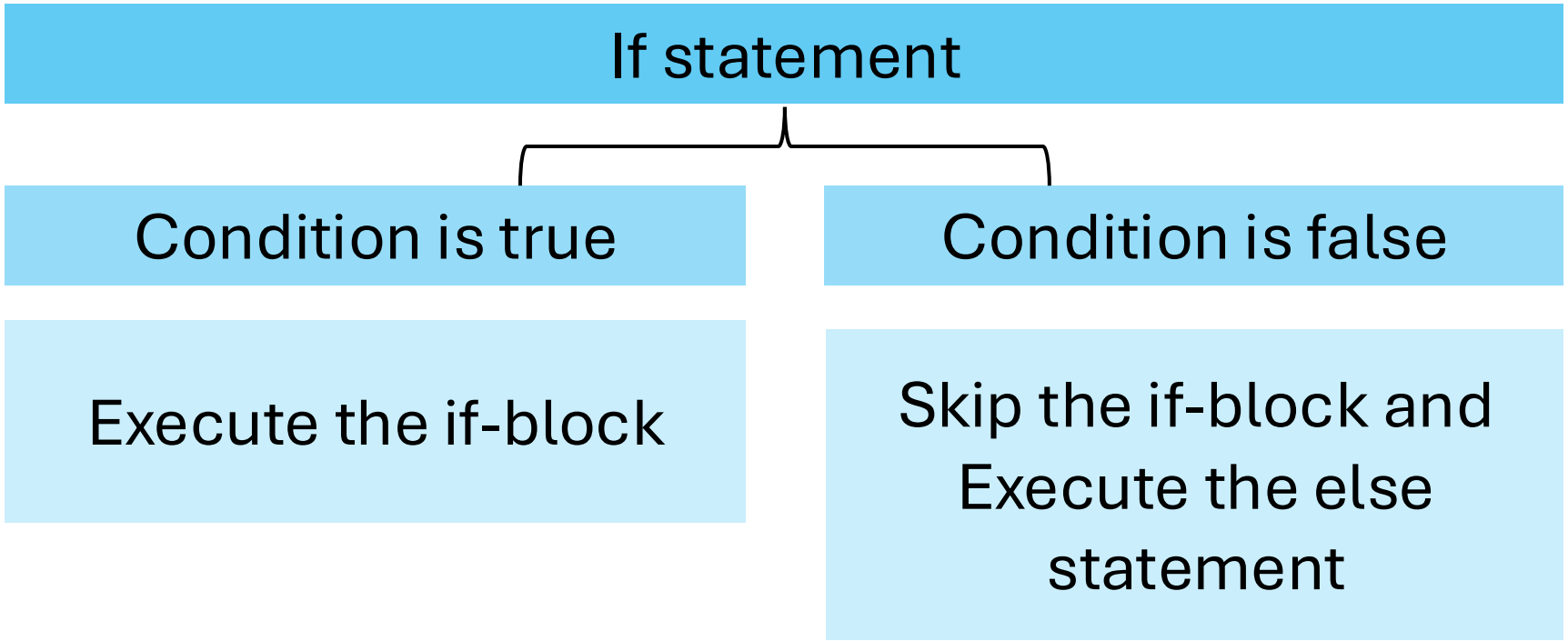
Condition: year >= 2000

If True → message is printed

If False → nothing happens

The **else** statement

- We can use an `else` statement to tell Python what code to run if the condition evaluates to `False`.
- An `else` statement must always be paired with an `if` statement, but as we have seen, `if` statements do not need to be paired with `else`.



The **else** statement

- We can use an `else` statement to tell Python what code to run if the condition evaluates to `False`.
- An `else` statement must always be paired with an `if` statement, but as we have seen, `if` statements do not need to be paired with `else`.

```
# Example of if/else statement

year = 1999
if year >= 2000:
    print('We are in the 21st century')
else:
    print('We are not in the 21st century')
```

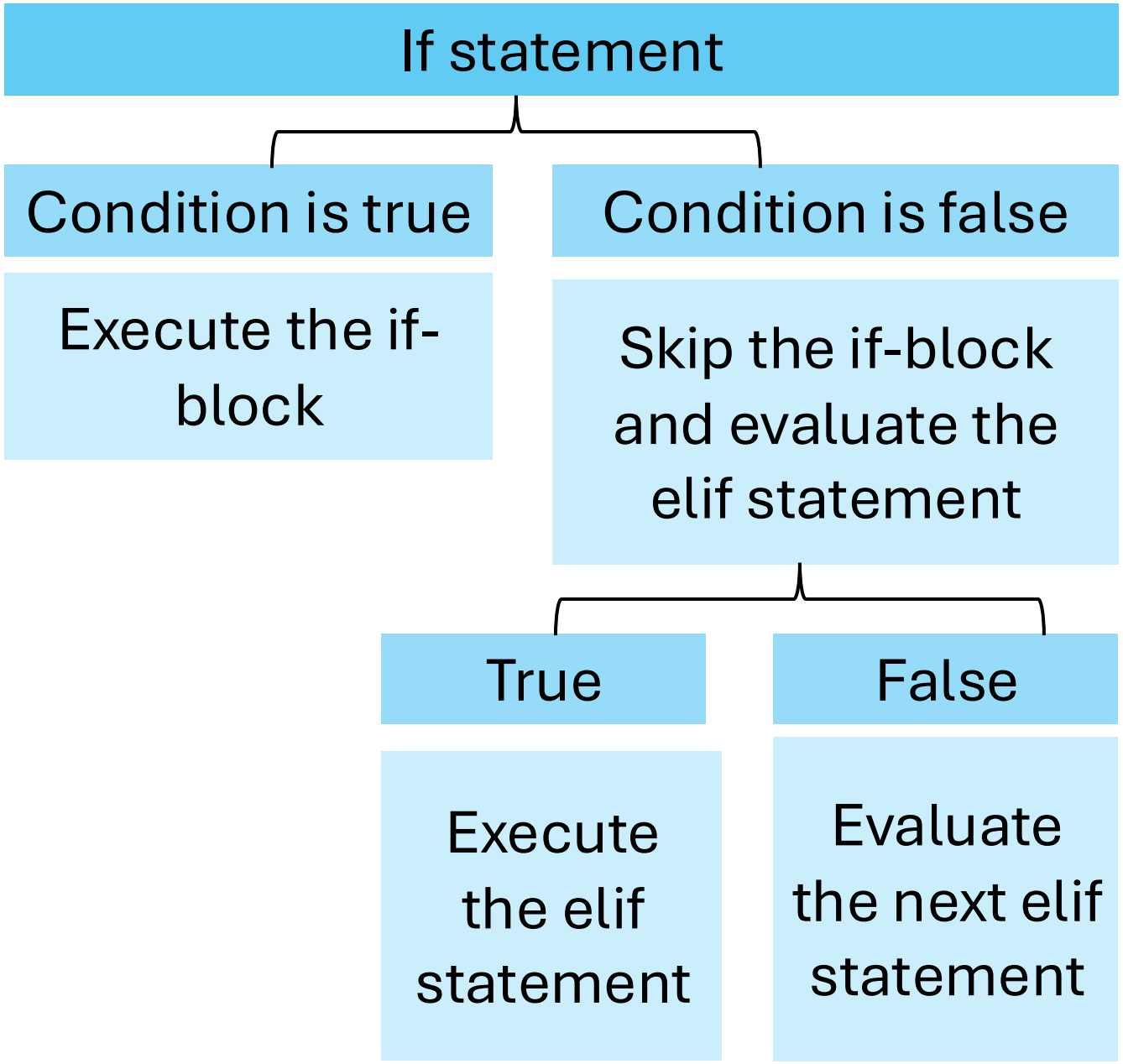
Condition: year >= 2000

If True → execute if statement

If False → execute else statement

The **elif** statement

- We can evaluate several conditions one after another with `elif`, which is short for "**else if**".
- Conditions are checked in the order they appear.
- Python will execute the code block under the first `True` condition and skip subsequent `elif` and `else` statements after without evaluating them.



The **else** statement

- We can evaluate several conditions one after another with `elif`, which is short for "**else if**".
- Conditions are checked in the order they appear.
- Python will execute the code block under the first `True` condition and skip subsequent `elif` and `else` statements after without evaluating them.

```
# Example of if/elif/else statement
year = 1867
if year >= 2000:
    print('We are in the 21st century')
elif year >= 1900:
    print('We are in the 20st century')
else:
    print('We are not in the 20th or 21st century')
```

Quiz: Conditional statements

1. **What keyword is used to check an additional condition after an if statement?**

A. Else if
B. elseif
C. elif
D. If else

Quiz: Conditional statements

2. How many *else* blocks can an if statement have?

A. Zero
B. One
C. Two
D. Unlimited

Quiz: Conditional statements

3. Which statement is true?

A. `elif` must come before `if`

B. `else` can appear before `elif`

C. An `if` statement can exist without `else`

D. An `if` statement must always have `elif`

Quiz: Conditional statements

4. **What is printed from the following code?**

```
x = 10

if x > 5:
    print('A')
else:
    print('B')
```

Quiz: Conditional statements

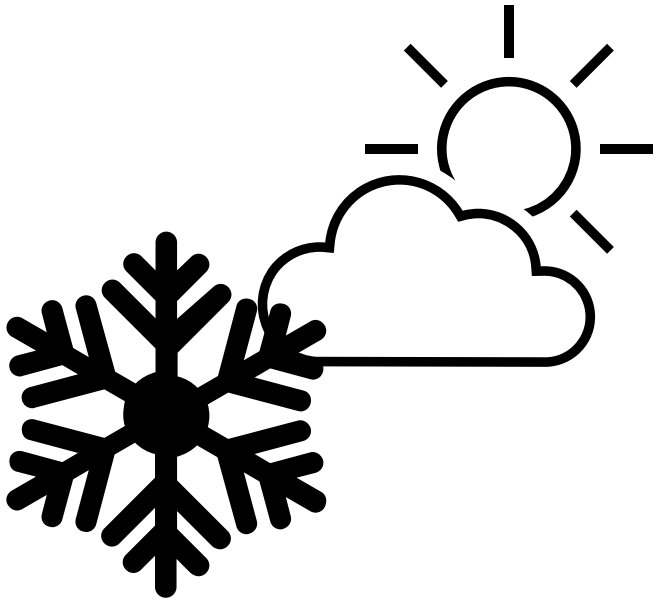
5. What is printed from the following code?

```
score = 85

if score >= 90:
     print('A')
elif score >= 80:
     print('B')
elif score >= 70:
     print('C')
else:
     print('D')
```

Exercise # 1

- Try this on your own:
  - Write a program where it outputs how cold the weather is based on these conditions:
    - Less than 0C, it is freezing cold!
    - Between 1 and 15C, it is cold!
    - Between 16 and 30C, it is hot!
    - Between 31 and 45C, it is really hot!
    - Over 45C, it is extremely hot!



Building more complex conditions

- We can use logical operators to check more complex conditions
- This allows us to test multiple rules at the same time
- For example, we can use **`and`** to **check if <u>both</u> conditions are True**

```
# Example of if statement with and
age = 20
has_id = True

if age >= 18 and has_id:
    print('You are able to enter')
```

Both conditions must be true to execute the indented code

Building more complex conditions

- We can use logical operators to check more complex conditions
- This allows us to test multiple rules at the same time
- For example, we can use **`or`** to **check if <u>either</u> conditions are True**

```
# Example of if statement with or
is_student = False
has_coupon = True

if is_student or has_coupon:
    print('You get a discount!')
```

One condition must be true to execute the indented code

Building more complex conditions

- We can use logical operators to check more complex conditions
- This allows us to test multiple rules at the same time
- For example, we can use **`not`** to **reverse a Boolean value**

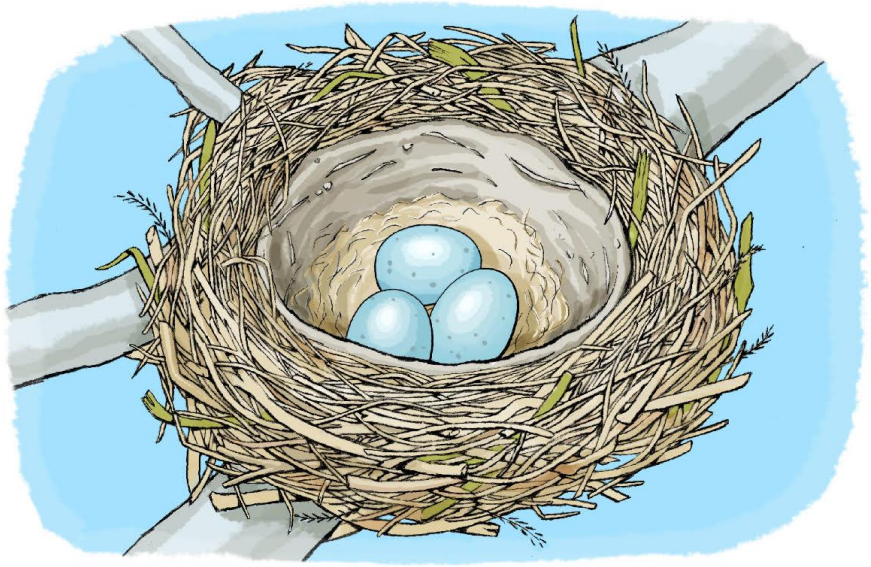
```
# Example of if statement with not
is_raining = False

if not is_raining:
    print('You can go outside!')
```

reverse the Boolean False to True so code executes

Nested conditions

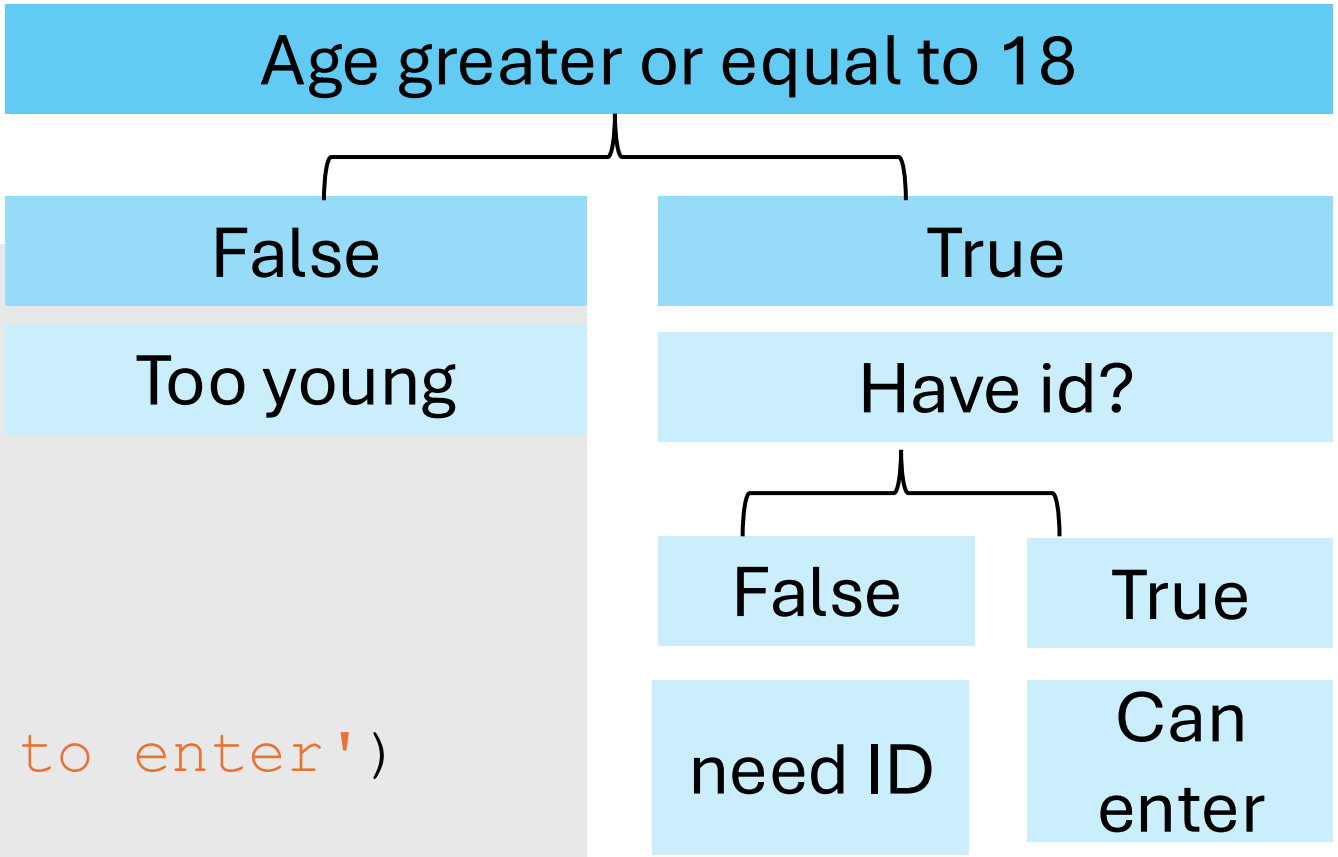
- **Nested conditionals** are if statements inside other if statements
- They allow us to check conditions **step by step**
- This is another way to handle more complex logic
- Whether to use logical operators, nested conditionals, or a mix of both depends on:
  - Personal preference
  - Readability
  - What exactly you are trying to check



Nested conditions

```
# Example
age = 20
has_id = True

if age >= 18:
  if has_id:
      print('You are able to enter')
  else:
      print('You need an ID')
else:
    print('You are too young')
```



The second *if* **only runs if the first condition is True**

Quiz: Nested conditions

1. **When does the inner if statement run in a nested condition?**

A. Always

B. Only if the outer `if` condition is true

C. Only if the outer `if` condition is false

D. Only if there is no elif

Quiz: Nested conditions

2. **In nested conditions, Python evaluates all if statements, even after one is true.**

A. True

B. False

Quiz: Nested conditions

3. What is printed from the following code?

```
age = 15

if age >= 18:
     print('Adult')
else:
     if age >= 13:
          print('Teen')
      else:
          print('Child')
```

Quiz: Nested conditions

4. What is printed from the following code?

```
score = 60

if score >= 50:
    if score >= 75:
        print('Distinction')
    else:
        print('Pass')
else:
    print('Fail')
```

Collection of values

- So far, we've worked with individual values:
  - Integers (int)
  - Floating-point numbers (float)
  - Strings (str)
  - Booleans (bool)
- However, in real programs, we often want to work with groups of values at the same time.

Why do we need collections?

- Collections allow us to:
  - Store many values in a single variable
  - Organize related data
  - Loop over values
  - Perform operations on groups of data
- Instead of creating many separate variables, we can store values together.

Collections in Python

- Python provides several built-in data types for working with multiple values that we will cover together!
  - **Lists** → ordered, changeable
  - **Tuples** → ordered, not changeable
  - **Sets** → unordered, no duplicates
  - **Dictionaries** → key–value pairs

Lists

- A **list** stores multiple values in a single variable
- Lists are created using **square brackets** []

```
# Example of a list
vowels =['a', 'e', 'i', 'o', 'u']
```

- We can also create an empty list by using the [ ] with nothing in them!

```
# Example of an empty list
empty_list =[]
```

Lists

- Lists can contain:
  - Numbers
  - Strings
  - Booleans
  - a mix of types
- We can even store lists within lists!

```
# Examples
scores = [90, 80, 82, 91, 80]
grades =['K', 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
foods = [
    ["apple", "banana", "orange"],       # fruits
    ["carrot", "broccoli", "spinach"]     # vegetables
]
```

Accessing items by index in a list

- Lists are **ordered**, meaning each item has a **position** (index)
- Indexing allows you to **retrieve specific items**
- Similar to strings: `string[0]` gets the first character
- **Slicing extracts multiple items at once**
  - Syntax: list[start:end] (includes start, excludes end)

```
# Accessing items
fruits = ["apple", "banana", "orange"]
fruits[0] #apple
fruits[0:2] #apple, banana
```

List membership and basic operations

- We can check if a value is in a list with the **`in`** operator

```
vowels =['a', 'e', 'i', 'o', 'u’]
'e' in vowels #check if e is in the vowels list
```

- We can check the length of a list

```
len(vowels) #5 items in the list
```

- The `+` and `*` operators also work on lists
  - + concatenates two lists, * repeats the list x amount of times

```
2 * vowels
#['a', 'e', 'i', 'o', 'u', 'a', 'e', 'i', 'o', 'u']
```

Mutating lists

- Lists are **mutable**, meaning they can be modified.
- To mutate or change a value in a list, we access its index and assign the new value

```
# A list can be changed in place
fruits = ["apple", "banana", "orange"]

# Change the second item
fruits[1] = "kiwi"

# Now list is apple, kiwi, orange
```

List methods

- Like strings, lists have their own methods.
- Almost all list methods allow us to mutate them

1. Adding items to the end of a list using the `append` method

```
rainbow = ['red', 'orange', 'yellow', 'green',
'light blue', 'blue', 'violet']

rainbow.append('purple')

#now rainbow is: 'red', 'orange', 'yellow',
'green', 'light blue', 'blue', 'violet', 'purple'
```

List methods

- Like strings, lists have their own methods.
- Almost all list methods allow us to mutate them

2. `Extend` method takes each item from the other list and adds it to the original list one by one.

```
rainbow = ['red', 'orange', 'yellow', 'green', 'light
blue', 'blue', 'violet']

rainbow.extend(['magenta', 'pink'])

#now rainbow is: 'red', 'orange', 'yellow', 'green',
'light blue', 'blue', 'violet', 'magenta', 'pink'
```

List methods

- Like strings, lists have their own methods.
- Almost all list methods allow us to mutate them

2. `Extend` method takes each item from the other list and adds it to the original list one by one. Note, passing in a string breaks it up into single characters

```
rainbow = ['red', 'orange', 'yellow', 'green', 'light
blue', 'blue', 'violet']

rainbow.extend('pink')

#now rainbow is: 'red', 'orange', 'yellow', 'green',
'light blue', 'blue', 'violet', 'p', 'i', 'n', 'k'
```

List methods

- Like strings, lists have their own methods.
- Almost all list methods allow us to mutate them

3. Adding items somewhere else into a list using the `insert` method and specifying the index to insert data into

```
rainbow = ['red', 'orange', 'yellow', 'green',
'light blue', 'blue', 'violet']

rainbow.insert(6, 'indigo') #add at position 6

#now rainbow is: 'red', 'orange', 'yellow',
'green', 'light blue', 'blue', 'indigo', 'violet'
```

List methods

- Like strings, lists have their own methods.
- Almost all list methods allow us to mutate them

3. Remove items using the `remove` method. Note this only gets rid of the first match

```
rainbow = ['red', 'orange', 'yellow', 'yellow',
'green', 'light blue', 'blue', 'violet']

rainbow.remove('yellow') #remove the first yellow

#now rainbow is: 'red', 'orange', 'yellow',
'green', 'light blue', 'blue', 'violet'
```

List methods

- Like strings, lists have their own methods.
- Almost all list methods allow us to mutate them

3. Remove items using the `remove` method. Note this only gets rid of the first match. To remove one or more item by index, we use the `del` operator

```
rainbow = ['red', 'orange', 'yellow', 'green',
'light blue', 'blue', 'violet']

del rainbow[0:3] #remove the first three colors

#now rainbow is: 'green', 'light blue', 'blue',
'violet'
```

Sorting lists

- Since lists are ordered, we can sort them. There are two main ways:

1. Sort in place (**modifies the original list** using the `sort` method)

```
numbers = [4, 1, 7, 3]
numbers.sort()    # sorts the list ascending
numbers # [1, 3, 4, 7]
```

```
fruits = ['pineapple', 'apple', 'kiwi', 'banana']
fruits.sort() # sorts the list alphabetically
fruits #apple, banana, kiwi, pineapple
```

Sorting lists

- Since lists are ordered, we can sort them. There are two main ways:

2. Create a new sorted list (**leaves the original list unchanged** using `sorted`)

```
numbers = [4, 1, 7, 3]
sorted_numbers = sorted(numbers)
numbers # remain the same [4, 1, 7, 3]
sorted_numbers #new sorted list
```

Tip: Use sorted() when you want a new list and don't want to touch the original.
Use .sort() when you want to change the list directly.

Quiz: Lists

1. **Which of the following correctly creates a list?**

A.
```
numbers = (1,2,3)
```

B.
```
numbers = [1,2,3]
```

C.
```
numbers = {1,2,3}
```

D.
```
numbers = '1,2,3'
```

Quiz: Lists

2. What is the index of the first element in a Python list?

A. 0

B. 1

C. -1

D. Depends on the list

Quiz: Lists

3. What does `my_list[-1]` return?

A. The first element

B. The second element

C. The last element

D. An error

Quiz: Lists

```
nums = [10, 20, 30, 40, 50]
```

4. What is the output of `nums[1:4]` ?

5. What is the output of `nums[:3]` ?

Tuples

Tuples are useful when values should stay the same

- Like lists, **tuples** are built-in data types that are ordered collection of values
- We can store multiple values, access them by indexing, slice them etc.
- Key difference: **<u>NOT MUTABLE (They cannot be changed once they are created).</u>**
  - We cannot update a tuple to add, remove, replace or reorder items
  - Think of them as "read-only"
- Tuples are created using **parentheses** ()

```
# Example of a tuple
coordinates = (43.7, -79.4)
```

- To create an empty tuple, we use () (Though, we can't add anything to it later)

```
# Example of an empty tuple
empty_tuple = ()
```

Tuples

Tuples are useful when values should stay the same

- Functions that work on lists and do not modify them in place also work on tuple
  - Example, length of tuple (`len`)

```
coordinates = (43.7, -79.4)
len(coordinates) #2
```

  - Example, `sorted`

```
sorted(coordinates) #[-79.4, 43.7]
```

Quiz: Tuples

1. **Which of the following correctly creates a tuple?**

A.
```
numbers = (1,2,3)
```

B.
```
numbers = [1,2,3]
```

C.
```
numbers = {1,2,3}
```

D.
```
numbers = '1,2,3'
```

Quiz: Tuples

2. What is the main difference between a list and a tuple?

A. Tuples can store only numbers

B. Tuples are mutable

C. Tuples are immutable

D. Tuples use square brackets

Quiz: Tuples

4. What happens if you try to run this code?

```
nums = (10, 20, 30, 40, 50)
nums[0] = 100
```

Set

- A **set** stores unique values
- Sets are **unordered** and **do not allow duplicates**
  - Because the elements do not maintain any specific order, and the is not guaranteed to be the same as the order in which you inserted them.
- They are created using curly braces {}

```
# Examples of a set
unique_numbers = {1, 2, 3, 3, 3}
unique_numbers #result is 1,2,3

things = {'coat', 'lock', 'box', 'book', 'apple',
'hair','xylophone', 'lock', 'book'}
things #result is apple, book, box, coat, hair,
lock, xylophone
```

Set

- We can turn a list into a set to remove duplicates using the `set()` function

```
visitor_post_codes = ['M5R', 'M5V', 'M1M', 'M1M', 'M1T']
set(visitor_post_codes) #{'M1M', 'M1T', 'M5R', 'M5V'}
```

- To create an empty set, we use the `set()` function

```
# Example of an empty set
empty_set = set()
```

Note. because sets are unordered, we cannot slice them or access items by index

Set

- Sets are mutable, so we can add and remove items, and the set will be modified in place.
- If an item is already in a set, it won't be duplicated

```
things = {'coat', 'lock', 'box', 'book', 'apple',
'hair','xylophone', 'lock', 'book'}
things.add('lock') # the set will not update
```

- We can also use remove, check for membership etc.

```
things.remove('apple') # result {'book', 'box',
'coat', 'hair', 'lock', 'mirror', 'xylophone'}
```

Set operations

- Sets support operations that are **specific to collections of unique items**. These include:

1. **Union** → combines two sets to include <u>all unique values from both</u>

```
rainbow = {'red', 'orange', 'yellow', 'green',
'blue', 'indigo', 'violet'}
olympic_flag = {'red', 'green', 'yellow', 'blue',
'black'}

rainbow.union(olympic_flag)
#{'black', 'blue', 'green', 'indigo', 'orange',
'red', 'violet', 'yellow'}
```

Set operations

- Sets support operations that are **specific to collections of unique items**. These include:

2. **Intersection** → finds values <u>common to both sets</u>

```
rainbow = {'red', 'orange', 'yellow', 'green',
'blue', 'indigo', 'violet'}
olympic_flag = {'red', 'green', 'yellow', 'blue',
'black'}

rainbow.intersection(olympic_flag)
#{'blue', 'green', 'red', 'yellow'}
```

Set operations

- Sets support operations that are **specific to collections of unique items**. These include:

3. **Symmetric difference** → finds values <u>in either but **not both** sets</u>

```
rainbow = {'red', 'orange', 'yellow', 'green',
'blue', 'indigo', 'violet'}
olympic_flag = {'red', 'green', 'yellow', 'blue',
'black'}

rainbow.symmetric_difference(olympic_flag)
#{'black', 'indigo', 'orange', 'violet'}
```

Set operations

- Sets support operations that are **specific to collections of unique items**. These include:

4. **Difference** → finds values **<u>in the first set</u>** <u>but</u> **<u>not the second</u>** <u>set</u>

```
rainbow = {'red', 'orange', 'yellow', 'green',
'blue', 'indigo', 'violet'}
olympic_flag = {'red', 'green', 'yellow', 'blue',
'black'}

rainbow.differnce(olympic_flag)
#{'orange', 'indigo', 'violet'}
olympic_flag.differnce(rainbow)
#{'black'}
```

Quiz: Sets

1. **Which of the following correctly creates a set?**

A.
```
numbers = (1,2,3)
```

B.
```
numbers = [1,2,3]
```

C.
```
numbers = {1,2,3}
```

D.
```
numbers = '1,2,3'
```

Quiz: Sets

2. What is a key property of a set?

A. Elements are ordered

B. Elements can be duplicated

C. Elements can be accessed by indexing

D. Elements are unique

Quiz: Sets

3. When is it useful to use a set instead of a list?

A. When you need to maintain the order of elements.

B. When you want to store duplicate elements.

C. When you need unique elements.

D. When you need to access elements by index.

Quiz: Sets

```
A = {1,2,3}
B = {3,4,5}
```

4. What is the output of `A.union(B)`?

5. What is the output of `A.intersection(B)`?

6. What is the output of `A.difference(B)`?

Dictionaries

- Store data in **key:value** pairs and let us look up values by keys
- Ordered and mutable
- Every key in a dictionary is unique
- To make a dictionary, we use curly braces {} around the key value pair
- **Keys are immutable** data types (strings, numbers, Booleans or tuples)

```
# Examples of a dictionary
capitals = {'Canada': 'Ottawa',
'United States': 'Washington, D.C.',
'Mexico': 'Mexico City'}
```

- To create an empty dictionary, we use curly braces

```
# Example of an empty dictionary
empty_dictionary = {}
```

Accessing, adding and updating dictionaries

- We can access the contents of a dictionary by **looking up the key using [ ]**

```
# Example
capitals['Canada'] #returns 'Ottawa'
```

- Trying to access a key that doesn’t exist will result in an error

```
# Example
capitals['Italy'] #returns KeyError: 'Italy'
```

Accessing, adding and updating dictionaries

- The `get()` method provides a safe way to work with dictionary values.
  - Takes the name of the key to look up and returns the corresponding value of the key exists, otherwise a default value without modifying the dictionary

```
# Example
capitals.get('Italy', 'Not Found')
#returns 'Not Found'
```

- We can also check to see if a key is in a dictionary with `in`

```
# Example
'United States' in capitals # True
```

  - Only looks for keys, not values

```
# Example
'Washington, D.C.' in capitals # False
```

Accessing, adding and updating dictionaries

- If we assign a value to a key that doesn't exist, the **key:value pair will be added** to the dictionary.

```
capitals['Italy'] = 'Milan'
capitals # {'Canada': 'Ottawa', 'United States':
'Washington, D.C.', 'Mexico': 'Mexico City',
'Italy': 'Milan'}
```

- If we assign a value to a **key that already exists, the value for that key will be updated.**

```
# Oops! It’s actually Rome
capitals['Italy'] = 'Rome'
capitals # {'Canada': 'Ottawa', 'United States':
'Washington, D.C.', 'Mexico': 'Mexico City',
'Italy': 'Rome'}
```

Accessing, adding and updating dictionaries

- To remove a key:value pair from a dictionary, we use the `del` operator

```
del capitals['Mexico']
capitals # {'Canada': 'Ottawa', 'United States':
'Washington, D.C.', 'Italy': 'Milan'}
```

Dictionary methods

- Python dictionaries have methods for getting keys, values, and items (that is, key:value pairs).
- This is useful for getting checking for values in a dictionary, and working with keys, values, and items one-by-one.
- Example, we get identify all keys in the dictionary

```
capitals.keys()
# dict_keys(['Canada', 'United States', 'Italy'])
```

- We can use conditionals to check if a value exists in a dictionary.

```
if 'Ottawa' in capitals.values():
     print('Ottawa is a capital')
else:
     print('Ottawa is not a capital')
```

Quiz: Dictionaries

1. **Given the dictionary:**

```
student = {"name": "Alice", "age": 20,
"major": "Physics"}
```

How would we access Alice's major?

A. student.major

B. student["major"]

C. student.get("Alice")

D. student[0]

Quiz: Dictionaries

2. What would this code do:

```
student = {"name": "Alice", "age": 20}
student["age"] = 21
```

A. Add a new key “age” with value of 21

B. Update the value of ”age” to 21

C. Raise an error

Quiz: Dictionaries

3. Which of the following would return a list of all keys

A. `dict.keys()`

B. `dict.values()`

C. `dict.items()`

D. `dict.get()`

Quiz: Dictionaries

4. What happens when we try to access a key that doesn’t exist?

A. Returns None

B. Returns a 0

C. Returns a KeyError

D. Automatically creates the key

Quiz: Dictionaries

5. Fill in the blank to add a new key-value pair “grade”: “A” to the dictionary

```
student = {"name": "Alice", "age": 20}
student[__] = ___
```

6. Fill in the blank to get the value of “name” from the dictionary

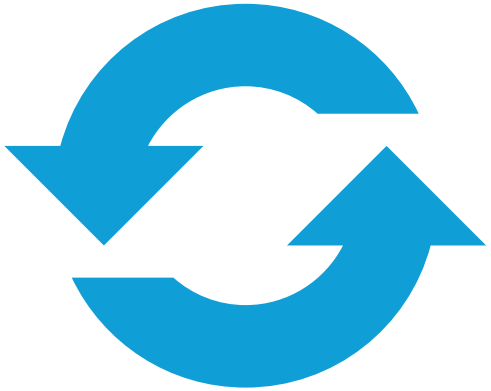
```
student = {"name": "Alice", "age": 20}
student.___("name")
```

Summary table

Type	Mutable	Ordered	Use when …
List	Yes	Yes	Use a list when you want to keep track of an **ordered sequence of items** and may need to **update, add, or remove elements**.
Tuple	No	Yes	Use a tuple when you want an **ordered sequence that should not change**.
Set	Yes	No	Use a set when you want to **store unique values**, the **order doesn't matter**, and the elements are **immutable**.
Dict	Yes	No	Use a dictionary to **map keys to values**. Keys must be **immutable**, while values can be any type

What are iteration and loops?

- Sometimes we need to **repeat code** multiple times or process **each item in a collection**.



- Copy/pasting is inefficient and error-prone → use **loops**.

- Python Loops
  1. **for loops** – repeat code for each item in an iterable (list, tuple, set, dictionary, string, etc.).
  2. **while loops** – repeat code while a condition is True.

For loops

- A for loop **repeats a block of code** for every item in a collection (called an **iterable**).
  You need a **variable name** to refer to each item while looping.
  - Use a name that **makes sense** (fruit, number, word)
  - If unsure, i is a common choice.

```
vowels =['a', 'e', 'i', 'o', 'u']
for vowel in vowels:
     print(vowel)
a
e
i
o
u
```

For loops

- A for loop **repeats a block of code** for every item in a collection (called an **iterable**).
  You need a **variable name** to refer to each item while looping.
  - Use a name that **makes sense** (fruit, number, word)
  - If unsure, i is a common choice.

```
vowels =['a', 'e', 'i', 'o', 'u']
for vowel in vowels:
     print(vowel)
a
e
i
o
u
```

For loops

- A for loop **repeats a block of code** for every item in a collection (called an **iterable**).
  You need a **variable name** to refer to each item while looping.
  - Use a name that **makes sense** (fruit, number, word)
  - If unsure, i is a common choice.

```
vowels =['a', 'e', 'i', 'o', 'u']
for vowel in vowels:
     print(vowel)
a
e
i
o
u
```

For loops

- A for loop **repeats a block of code** for every item in a collection (called an **iterable**).
  You need a **variable name** to refer to each item while looping.
  - Use a name that **makes sense** (fruit, number, word)
  - If unsure, i is a common choice.

```
vowels =['a', 'e', 'i', 'o', 'u']
for vowel in vowels:
     print(vowel)
a
e
i
o
u
```

For loops

- A for loop **repeats a block of code** for every item in a collection (called an **iterable**).
  You need a **variable name** to refer to each item while looping.
  - Use a name that **makes sense** (fruit, number, word)
  - If unsure, i is a common choice.

```
vowels =['a', 'e', 'i', 'o', 'u']
for vowel in vowels:
     print(vowel)
a
e
i
o
u
```

For loops

- If we wanted to simply run a block of code n number of times, we use the `range()` function to create an iterable to loop over

```
for i in range(5):
     print(i, i*2)
0 0
1 2
2 4
3 6
4 8
```

Recall: The range() function generates a sequence of numbers. The stop number is not included.

For loops

- We can loop through lists and create new modified ones!

```
input_files = ['data_01.csv', 'data_02.csv', 'data_03.csv',
'data_04.csv']
output_files = []

for i in input_files:
   output_file_name = 'processed_' + i.replace('.csv', '.xlsx')
   output_files.append(output_file_name)

output_files # ['processed_data_01.xlsx',
'processed_data_02.xlsx',
'processed_data_03.xlsx',
'processed_data_04.xlsx']
```

Quiz: For loops

1. **What does the following code print?**

```
fruits =['apple', 'banana', 'cherry']
for fruit in fruits:
     print(fruit)
```

A. fruits

B. [“apple”, ”banana” ,”cherry”]

C. apple banana cherry each a new line

D. Error

Quiz: For loops

2. What does the following code print?

```
for i in range(3):
    print(i)
```

A. 1 2 3

B. 0 1 2

C. 0 1 2 3

D. Error

Quiz: For loops

3. What does the following code print?

```
for i in range(1,5):
     print(i*2)
```

A. 2 4 6 8

B. 1 2 3 4

C. 1 3 5 7

D. Error

Quiz: For loops

4. Fill in the blank to iterate through this list

```
numbers =[10, 20, 30]
for __ in numbers:
    print(num)
```

5. Fill in the blank to print numbers from 1 to 5

```
for i in ____:
    print(i)
```

Looping with multiple values

- Sometimes, we want to **loop over more than one value at a time**.
- This happens with:
  - enumerate() → gives **index and value**
  - dict.items() → gives **key and value**
- Since these methods return pairs of values, we use two variables in the loop.
- Python “unpacks” the pair into the two variables automatically.

Looping with multiple values

1. using `enumerate()`

```
stops = ['Sheppard-Yonge', 'Bayview', 'Bessarion', 'Leslie',
'Don Mills']

for idx, stop in enumerate(stops):
     print(f'Stop {idx + 1} is {stop}.')

Stop 1 is Sheppard-Yonge.
Stop 2 is Bayview.
Stop 3 is Bessarion.
Stop 4 is Leslie.
Stop 5 is Don Mills.
```

enumerate() gives two values for each item: the index and the value itself. Python automatically “unpacks” these into the variables you provide in the loop.

Looping with multiple values

1. using `enumerate()`

bus stops

0. Sheppard-Young
1. Bayview
2. Bessarion
3. Leslie
4. Don Mills

enumerate

index	value
0	Sheppard-Young
1	Bayview
2	Bessarion
3	Leslie
4	Don Mills

loop unpacks:

idx	stop
index	value

Stop 1 is Sheppard-Yonge.
Stop 2 is Bayview.
Stop 3 is Bessarion.
Stop 4 is Leslie.
Stop 5 is Don Mills.

Looping with multiple values

2. using `dict.items()`

```
prices = {"apple": 2, "banana": 1, "orange": 3}

for fruit, price in prices.items():
    print(fruit, price)

apple 2
banana 1
orange 3
```

items() gives two values for each item: the key and the value itself. Python automatically “unpacks” these into the variables you provide in the loop.

Looping with multiple values

2. using `dict.items()`

Prices

apple : $2
banana: $1
orange: $3

items

Key	value
apple	2
banana	1
orange	3

loop unpacks:

fruit	price
index	value

apple 2
banana 1
orange 3

Quiz: For loops with multiple values

1. Which of the following is the correct syntax for iterating over a dictionary

A.
```
for key, value in my_dict:
```

B.
```
for key, value in my_dict.items():
```

C.
```
for key in my_dict.items():
```

D.
```
for key.value in my_dict():
```

Quiz: For loops with multiple values

2. What does the following code print?

```
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

A. apple banana cherry

B. 0 apple 1 banana 2 cherry

C. 1 apple 2 banana 3 cherry

D. Error

Quiz: For loops with multiple values

3. Fill in the blank to loop through a list with index and value

```
names = ['Alice', 'Bob', 'Charlie']
for __,__ in enumerate(names):
    print(index, name)
```

4. Fill in the blank to iterate through a dictionary and print keys and values

```
student = {'name': 'Alice', 'age': 20}
for __,__ in student.items():
    print(key, value)
```

Nested loops

- What is a Nested Loop?
  - A loop inside another loop
  - Outer loop → controls the first set of repetitions
  - Inner loop → runs completely for each iteration of the outer loop

```
for i in range(1, 3):          # outer loop = 1
    for j in range(1, 3):      # inner loop = 1
        print(i, "*", j, "=", i*j)

1 * 1 = 1
```

Nested loops

- What is a Nested Loop?
  - A loop inside another loop
  - Outer loop → controls the first set of repetitions
  - Inner loop → runs completely for each iteration of the outer loop

```
for i in range(1, 3):          # outer loop = 1
    for j in range(1, 3):      # inner loop = 2
        print(i, "*", j, "=", i*j)

1 * 1 = 1
1 * 2 = 2
```

Nested loops

- What is a Nested Loop?
  - A loop inside another loop
  - Outer loop → controls the first set of repetitions
  - Inner loop → runs completely for each iteration of the outer loop

```
for i in range(1, 3):          # outer loop = 2
    for j in range(1, 3):      # inner loop = 1
        print(i, "*", j, "=", i*j)

1 * 1 = 1
1 * 2 = 2
2 * 1 = 2
```

Nested loops

- What is a Nested Loop?
  - A loop inside another loop
  - Outer loop → controls the first set of repetitions
  - Inner loop → runs completely for each iteration of the outer loop

```
for i in range(1, 3):          # outer loop = 2
    for j in range(1, 3):      # inner loop = 2
        print(i, "*", j, "=", i*j)

1 * 1 = 1
1 * 2 = 2
2 * 1 = 2
2 * 2 = 4
```

Quiz: Nested for loops

1. What does the following code print?

```
for i in range(1, 4):
    for j in range(1, 3):
        print(i, j)
```

A. 1 2 3 4 5 6

B. 1 1 1 2 2 1 2 2 3 1 3 2

C. None of the above

D. Error

Quiz: Nested for loops

2. What is true about nested for loops?

A. The inner loop runs once for each iteration of the outer loop

B. The outer loop runs once for each iteration of the inner loop

C. Nested loops cannot be used with lists

D. Nested loops always produce a KeyError

While loops

- What if we aren't sure how many times code needs to run, but we want to know how to tell when we're done? In this case, we use a **while** loop!
- A while loop only runs the indented block of code when a condition is met

```
countdown = 4
while countdown > 0:
     print (countdown)
     countdown -= 1
4
3
2
1
```

Starts the counter at 4.
While loop keeps running the loop as long as the condition is True.
Prints the current value of countdown.
Subtracts 1 from countdown each time the loop runs.

Infinite loops

- Wait… what happens if we omit the last line of the code in the countdown example and the counter never changes? It'll keep printing 4 forever!
- That's called an **infinite** loop, which can crash your program or freeze it.

```
countdown = 4
while countdown > 0:
     print (countdown)
4
4
4
4
4
4…
```

To create an intentional infinite loop, we make the `while` condition `True`.

Break statements

- We use break statements to interrupt the execution of a loop

```
countdown = 4
while countdown > 0:
      print (countdown)
      if countdown == 3:
            break
      countdown -= 1
4
3
```

This is useful when you want to stop early based on a certain condition.
e.g. stops the loop immediately if countdown is 3.

Continue statements

- We can interrupt a loop with a continue statement, which tells python to leave the current iteration of the loop and start back at the top
- Useful when you want to ignore certain cases but keep looping.

```
for i in range(1, 6):
    if i % 2 == 0: # if the number is even
        continue  # skip the rest of the loop for this iteration
    print(i)
1
3
5
```

This example, skips the rest of the loop for the values that are divisible by 2, only printing odd numbers

Quiz: While loops

1. **What does the following code print?**

```
i = 1
while i <= 3:
     print (i)
     i += 1
```

A. 1 2 3 4

B. 1 2 3

C. 0 1 2 3

D. Infinite loop

Quiz: While loops

2. What happens in this scenario?

```
while True:
     print ('Hello')
```

A. Prints “Hello” once then stops
B. Causes an error
C. Runs infinitely
D. Prints ”Hello” 3 times

Quiz: While loop

3. Which of the following stops a loop, even if the condition is still True

A. `continue`

B. `break`

C. `pass`

D. `stop`

Quiz: While loop

4. What does the continue statement do

A. Stops the loop completely
B. Skips the rest of the current iteration and moves on to the next
C. Exits the loop
D. Prints “continue”

Quiz: While loops

5. Fill in the blank to write a while loop that prints numbers 1 to 5

```
i = 1
while ___ :
     print (i)
     i += 1
```

6. Fill in the blank to write a loop that stops when i reaches 4

```
i = 1
while True :
     i += 1
     print (i)
     if ____:
          break
```